

ChainTrace

From thousands of log lines to one attack story.

Product Requirements Document

Team member	Role	Owns
Bhanu Prasad	Lead / Detection Engineer	Parsers, detection rules, scoring, correlation, attack story, backend API, attack simulator, evaluation, integration, deployment
Mahadev H	Frontend + Data + Docs	React dashboard, all UI pages, baseline traffic generator, README, architecture diagram, slides, demo video

Version 1.2 (PostgreSQL) • Date: 4 October 2026 • Duration: 12 hours • Stack: Python, FastAPI, React

What changed in v1.2: storage is now **PostgreSQL** (schema in Section 5; engine still runs in memory, results are written to Postgres after each analysis; new GET /api/analyses for history; tests T18-T19). **v1.1:** explicit Emergency MVP fallback (Section 8); events sorted by time before baseline and correlation; R10 uses a small documented signature set on URL-decoded requests; R11 renamed “Possible large data transfer” (HTTP response bytes only); S4 redesigned so only /24 aggregation catches it; scenario detection rate is the headline metric; Incident Detail is the UI centerpiece; judge Q&A prep added.

Guiding principle: a simple solution that works is better than a complex solution that doesn't. Every feature in this document is chosen because it can be built, integrated and tested inside 12 hours. Anything that risks the core demo is cut.

01 Problem Statement

Official statement (ALG-CYBER-01): Thousands of security events can hide an attacker among normal activity. Build a system that identifies suspicious behavior from logs, identifies suspicious users/IPs, connects related events and creates an incident timeline.

The real problem behind it: a security analyst does not lack alerts — they drown in them. A server produces tens of thousands of log lines a day. A typical tool flags each suspicious line in isolation: “50 failed logins”, “sudo used”, “new user created”. Each alert on its own looks minor, so the analyst misses that these three alerts are actually **one attacker** breaking in, escalating privileges and leaving a backdoor.

Three specific gaps we target:

- **Isolation:** events are flagged one by one; nobody connects the brute force on an IP to the sudo command run later by a user account.
- **Quiet attacks are missed:** simple thresholds catch loud brute force but miss password spraying (one password, many users) and low-and-slow attacks spread over hours and IPs.
- **No proof:** alerts say “suspicious” without showing the exact raw log lines and the reason, so analysts cannot trust or verify them.

Must-have from PS	What the judges are checking
Log ingestion	Can it read real-world, messy logs?
Anomaly detection / rules	Does it actually find attacks, with few false positives?
Suspicious user/IP detection	Can it name the attacker entities?
Event grouping	Are related events grouped together?
Incident timeline	Is there a clear chronological picture?
Evidence	Can every claim be traced to raw logs?
Bonus	Explain the likely attack sequence, not isolated events

02 How We Are Going to Solve It

ChainTrace is a web app. The analyst uploads server logs (or runs a built-in attack simulation). ChainTrace parses every line, runs detection rules, scores each IP and user, links related alerts into incidents and writes a plain-English **attack story** for each incident, mapped to MITRE ATT&CK, with every step backed by the exact raw log lines.

The pipeline in 7 steps

#	Step	What happens	Output
1	Ingest	Upload auth.log (SSH/sudo) and web access.log (Apache/Nginx combined). Format auto-detected. Malformed lines skipped and counted, never crash.	Raw lines + line numbers
2	Normalize	Every line becomes one standard Event: time (UTC), source, type, IP, user, HTTP fields, raw text. Events from all files are merged and sorted by timestamp ; original file + line number are kept so evidence still points to the real line.	Time-sorted event list
3	Detect	11 deterministic rules (brute force, spraying, low-and-slow, login-after-failures, off-hours, new IP, sudo, account creation, web scanning, web payloads, possible large transfer). Simple per-user baseline for hours and known IPs.	Alerts with reason + evidence IDs

#	Step	What happens	Output
4	Score	Each alert adds points to its IP/user. More kill-chain stages = higher multiplier. Weak signals add up.	Risk score 0–100 per entity
5	Correlate	Alerts become graph nodes. Linked if they share IP, user or /24 subnet within 2 hours. IP→user pivot: a successful login links the attacking IP to everything that account does next.	Incidents
6	Explain	Alerts in an incident are sorted by time and mapped to kill-chain stages. A template generates the attack story and recommended response.	Attack story + timeline
7	Present	Dashboard: overview, incident list, incident detail with story, timeline and click-to-see evidence, entity table, evaluation page.	UI

Example output (what judges will see)

Incident #1 — Critical (risk 94): SSH compromise with persistence

Between 02:03 and 02:41, IP 185.220.101.7 made 214 failed SSH logins against 3 accounts (*Credential Access, T1110.001*). At 02:41 it logged in successfully as **deploy** — the first login for this account from this IP and outside its normal 09:00–18:00 hours (*Initial Access, T1078*). Within 4 minutes **deploy** ran sudo as root (*Privilege Escalation, T1548.003*) and created a new account **sysupdate** (*Persistence, T1136.001*).

Assessment: likely successful SSH compromise with a backdoor account.

Recommended: disable deploy and sysupdate, block 185.220.101.7, rotate credentials.

Scope

IN scope (we build this)	OUT of scope (we deliberately skip this)
Linux auth.log + web access log (combined format)	Windows Event Logs, firewall/netflow, cloud logs
Batch upload and analysis	Real-time streaming, Kafka, agents
11 explainable rules + per-user baseline	Deep learning, LLM in the detection path
Attack simulator with ground-truth labels	Attacking or scanning any real system
Single-user web app with PostgreSQL storage (analysis history)	App login system, multi-tenant, RBAC
One service (FastAPI serves React build) + one PostgreSQL	Docker-compose stacks, Elasticsearch, Redis, microservices

03 What Makes Our Solution Innovative

Most teams will build “upload logs → list of flagged IPs”. That satisfies the must-haves but not the bonus. The PS explicitly rewards explaining the attack sequence *instead of* flagging isolated events. Our innovations:

Innovation	Why it matters	Typical team
1. Attack stories, not alert lists	Incidents are written as a chronological narrative mapped to kill-chain stages and MITRE ATT&CK, with an assessment and recommended response.	Shows a table of alerts
2. IP→user pivot correlation	A successful login from an attacking IP links that IP to everything the account does afterwards. This connects network-side and account-side activity — the exact gap attackers exploit.	Groups only by IP

Innovation	Why it matters	Typical team
3. Catches quiet attacks	Password spraying (one IP, many users, few tries each) and low-and-slow brute force spread across a /24 subnet over hours — both slip under simple thresholds.	Only catches loud brute force
4. Built-in attack simulator with ground truth	Generates realistic normal traffic and injects chosen attacks with labels. Judges pick the attack live; we prove detection with precision/recall.	Hand-made demo file, no metrics
5. Evidence by design	Every alert stores the raw line numbers that triggered it. Click any story step → see the exact log lines and the rule reason.	“Suspicious” with no proof
6. Deterministic and explainable	No LLM guessing. Same input always gives the same output. Every decision traces to a named rule and threshold in one config file.	Black-box AI summary
7. Risk accumulation	Weak signals add up; incidents spanning more kill-chain stages get a higher multiplier, so a real breach outranks noisy scanning.	Every alert is equal

04 How It Addresses the Problem

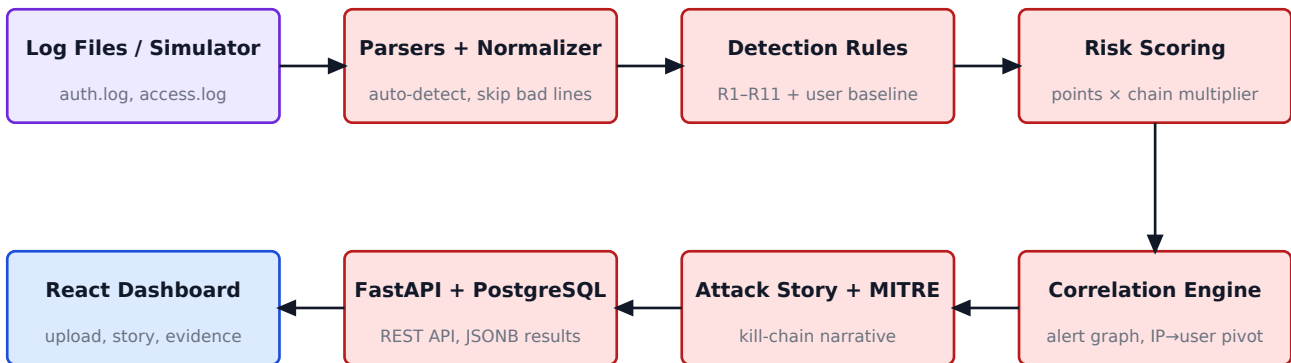
PS requirement	ChainTrace feature	Status
Log ingestion	Auto-detecting parsers for auth.log and access.log; tolerant of bad lines; skipped-line count shown in UI	P0
Anomaly detection / rules	11 rules + per-user baseline (normal hours, known IPs); optional Isolation Forest score (P2)	P0
Suspicious user/IP detection	Risk-scored entity table with search and filter by level	P0/P1
Event grouping	Correlation engine: alert graph with shared IP/user/subnet + time window + IP→user pivot	P0
Incident timeline	Vertical timeline per incident, ordered and colour-coded by kill-chain stage	P0
Evidence	Raw log lines with file name and line number behind every alert	P0
Bonus: attack sequence	Attack story generator with MITRE mapping, assessment and recommended response	P0

Mapping to the judging criteria

Criterion (weight)	How we score high
Functionality & Completion (30%)	All P0 features work end-to-end by Hour 8; live simulation demo proves it works on demand.
Technical Implementation (20%)	Clean pipeline architecture, typed schemas (Pydantic), single config for thresholds, clear module boundaries.
Innovation & Problem Understanding (20%)	Attack stories, IP→user pivot, quiet-attack detection, simulator with ground truth.
UX / Presentation (15%)	Story-first incident view, click-to-evidence, polished 3-minute demo with backup video.
Testing, Edge Cases & Reliability (15%)	Precision/recall on labelled data, zero-false-positive baseline test, malformed/empty/huge file tests, real-world Loghub sample.

05 System Design

Data flow: raw logs → normalized events → alerts → scored incidents → attack story → UI



 Bhanu (complex: engine + backend) Mahadev (frontend + data + docs) Shared (Bhanu attacks / Mahadev baseline)

Tech stack

Layer	Choice	Why (simple > complex)
Backend	Python 3.11, FastAPI, Uvicorn, Pydantic	Bhanu already knows FastAPI; Pydantic gives a typed contract
Detection	Plain Python (collections, re, datetime)	Fast, deterministic, no heavy dependencies
Optional ML (P2)	scikit-learn Isolation Forest	Only after all rules work
Storage	PostgreSQL 15+ via psycopg 3 (plain SQL, one schema.sql, JSONB for nested data)	Bhanu already knows Postgres; analyses survive restarts; JSONB keeps story/evidence simple; INET type for IPs
Frontend	React + Vite, Tailwind CSS, Recharts, React Router	Fast scaffold, ready-made charts
Testing	pytest (engine), manual UI checklist	Engine correctness is what judges probe
Deploy	One service on Render/Railway + managed Postgres (Neon / Supabase / Render)	One URL, no CORS issues; local Postgres as fallback

Repository structure

```
chaintrace/
├── backend/
│   ├── app/main.py           # FastAPI app, routes, serves frontend build (Bhanu)
│   ├── app/schemas.py       # Event, Alert, Incident, Entity models (Bhanu, frozen H1)
│   ├── app/db.py             # connection pool, save/load analysis, COPY (Bhanu)
│   ├── db/schema.sql         # all CREATE TABLE + indexes (Bhanu, H1)
│   ├── engine/parsers.py     # auth.log + access.log parsers, auto-detect (Bhanu)
│   ├── engine/rules.py       # R1-R11 (Bhanu)
│   ├── engine/baseline.py    # per-user normal hours / known IPs (Bhanu)
│   ├── engine/scoring.py     # risk scoring (Bhanu)
│   ├── engine/correlate.py   # alert graph + union-find -> incidents (Bhanu)
│   ├── engine/story.py       # attack story + MITRE + recommendations (Bhanu)
│   └── engine/evaluate.py    # precision / recall vs ground truth (Bhanu)
```

		config.yaml	# ALL thresholds in one place	(Bhanu)
		.env.example	# DATABASE_URL=postgresql://...	(Bhanu)
		tests/	# pytest	(Bhanu)
		simulator/		
		baseline.py	# normal users + web traffic	(Mahadev)
		attacks.py	# 5 attack scenarios + labels.json	(Bhanu)
		frontend/src/	# pages, components, api.js, mocks/	(Mahadev)
		docs/	# architecture.png, slides, demo script	(Mahadev)
		README.md		(Mahadev)

PostgreSQL design

Key rule: the engine runs entirely in memory; PostgreSQL only stores the finished result. Upload → parse → detect → correlate in Python → write everything in one transaction → API reads from Postgres. A database problem can never break detection, and the engine stays easy to unit-test without a DB.

```
CREATE TABLE analyses (
  id UUID PRIMARY KEY, created_at TIMESTAMPTZ DEFAULT now(),
  source TEXT CHECK (source IN ('upload','simulation')), files TEXT[],
  stats JSONB, has_ground_truth BOOLEAN DEFAULT false, evaluation JSONB);

CREATE TABLE events (
  analysis_id UUID REFERENCES analyses(id) ON DELETE CASCADE, id TEXT,
  ts TIMESTAMPTZ, source TEXT, type TEXT, src_ip INET, username TEXT, host TEXT,
  http JSONB, file TEXT, line_no INT, raw TEXT, PRIMARY KEY (analysis_id, id));

CREATE TABLE alerts (
  analysis_id UUID REFERENCES analyses(id) ON DELETE CASCADE, id TEXT,
  rule_id TEXT, severity TEXT, points INT, stage TEXT, mitre JSONB,
  src_ip INET, username TEXT, first_seen TIMESTAMPTZ, last_seen TIMESTAMPTZ,
  count INT, reason TEXT, evidence_event_ids TEXT[], PRIMARY KEY (analysis_id, id));

CREATE TABLE incidents (
  analysis_id UUID REFERENCES analyses(id) ON DELETE CASCADE, id TEXT,
  title TEXT, risk_score INT, level TEXT, entities JSONB, start_ts TIMESTAMPTZ,
  end_ts TIMESTAMPTZ, stages TEXT[], alert_ids TEXT[], summary TEXT,
  recommendation TEXT, story JSONB, PRIMARY KEY (analysis_id, id));

CREATE TABLE entities (
  analysis_id UUID REFERENCES analyses(id) ON DELETE CASCADE, type TEXT, value TEXT,
  risk_score INT, level TEXT, alert_count INT, first_seen TIMESTAMPTZ,
  last_seen TIMESTAMPTZ, PRIMARY KEY (analysis_id, type, value));

CREATE INDEX ON events (analysis_id, ts);
CREATE INDEX ON alerts (analysis_id, src_ip);
CREATE INDEX ON incidents (analysis_id, risk_score DESC);
```

- **Bulk insert with COPY** (psycopg 3 cursor.copy()), not row-by-row INSERT — 100k events must save in a couple of seconds.
- **Evidence lookup** = SELECT * FROM events WHERE analysis_id = \$1 AND id = ANY(\$2) using the alert's evidence_event_ids.
- **No ORM migrations** (no Alembic): one schema.sql, run once with psql. Column username avoids the reserved word user; the API still returns it as user.
- **Connection** from DATABASE_URL in .env; never commit credentials. Same code works for local Postgres and the managed one.

Data contract (frozen in Hour 1 — never change without telling the other person)

```
Event { id, ts (ISO-8601 UTC), source: "auth"|"web", type, src_ip, user, host,
  http: {method, path, status, bytes, ua} | null, file, line_no, raw }
type ∈ ssh_fail | ssh_success | invalid_user | sudo | user_add | group_add | http_request
```

```
Alert { id, rule_id, rule_name, severity: low|medium|high|critical, points,
      stage, mitre: {tactic, technique}, entity: {ip, user},
      first_seen, last_seen, count, reason, evidence_event_ids: [...] }

Incident { id, title, risk_score, level, entities: {ips:[...], users:[...]},
          start, end, stages:[...], alert_ids:[...], summary, recommendation,
          story: [ {ts, stage, text, alert_id} ] }

Entity { type: ip|user, value, risk_score, level, alert_count, first_seen, last_seen }
```

REST API

Method + path	Purpose	Returns
POST /api/analyze	Upload 1+ log files (multipart)	analysis_id + stats {lines_total, parsed, skipped, events, alerts, incidents}
POST /api/simulate	Body {scenarios:[...], seed}; generates logs and analyses them	Same as analyze + has_ground_truth: true
GET /api/analyses/{id}/summary	Overview cards + events-over-time histogram + top entities	Summary object
GET /api/analyses/{id}/incidents	Incident list sorted by risk	Incident[] (without story)
GET /api/analyses/{id}/incidents/{iid}	Full incident with story, alerts, evidence raw lines	Incident + Alert[] + Event[]
GET /api/analyses/{id}/entities	Risk-scored IPs and users	Entity[]
GET /api/analyses/{id}/evaluation	Precision/recall (simulated runs only)	Metrics object
GET /api/analyses	List previous analyses (newest first) — enabled by PostgreSQL	[{id, created_at, source, stats}]
GET /api/health	Liveness check incl. DB ping	{status: ok, db: ok}

06 Detection Engine Specification (Bhanu)

All thresholds live in **config.yaml** so they can be tuned in Hour 9 without touching code. Severity points: low 10, medium 20, high 35, critical 50.

ID	Rule	Logic (default threshold)	Sev.	Stage / MITRE	Pri.
R1	SSH brute force	≥10 failed logins from one IP within 5 min	High	Credential Access / T1110.001	P0
R2	Password spraying	One IP fails for ≥5 distinct usernames within 10 min	High	Credential Access / T1110.003	P0
R3	Low-and-slow brute force	≥15 failures over 6 h counted combined across one /24 subnet (or from one IP), where no single IP triggered R1	Med	Credential Access / T1110	P1
R4	Login after failures	Successful login from an IP with ≥5 failures in the previous 60 min	Crit	Initial Access / T1078	P0
R5	Off-hours login	Success outside the user's learned hours (default 08–20)	Med	Initial Access / T1078	P1
R6	New source IP	Success for a user from an IP never seen for that user in the baseline window	Low	Initial Access / T1078	P1

ID	Rule	Logic (default threshold)	Sev.	Stage / MITRE	Pri.
R7	Privilege use after suspicious login	sudo/su by a user within 60 min of an R4/R5/R6 login	High	Privilege Escalation / T1548.003	P0
R8	Account created / added to sudo	useradd, or usermod adding a user to sudo/wheel	Crit	Persistence / T1136.001, T1098	P0
R9	Web scanning	≥30 HTTP 404s from one IP in 5 min, or scanner user-agent (sqlmap, nikto, gobuster, dirbuster, nmap)	Med	Reconnaissance / T1595	P0
R10	Web attack payload	URL-decoded path/query matches the small signature set below. Reported as “matched signature”, never “confirmed”	High	Initial Access / T1190	P0
R11	Possible large data transfer	Sum of HTTP response bytes (access-log bytes field) to one IP in 10 min > 50 MB or > 20× median per-IP volume	High	Exfiltration (possible) / TA0010	P1

R10 signature set (keep it small and documented)

Category	Signatures (case-insensitive, matched after URL-decoding once)	UI label
SQL injection	' or ', union select, select ... from, sleep(, -- at end of parameter	Matched signature: SQL injection pattern
XSS	<script, javascript:, onerror=, onload=	Matched signature: XSS pattern
Path traversal	../, ..\, /etc/passwd (raw ../%2f is covered by decoding)	Matched signature: path traversal pattern
Command injection	;cat , cat , &&, \$(, `	Matched signature: command injection pattern

Evidence shows the matched signature and the decoded request. Wording rule for R11: the story says “*possible data exfiltration indicated by unusually large HTTP response volume*” — an access log shows response size, not network traffic.

Baseline (for R5, R6)

- Pipeline order: parse → normalize → **sort by timestamp** → baseline → rules → correlation. Baseline window = first 25% of the **time range** (not the first 25% of lines, which breaks on unordered or merged files). For each user: hours of successful logins and set of source IPs.
- If a user has fewer than 5 logins in the baseline, skip R5/R6 for that user (avoids false positives on rare users).

Risk scoring

- Entity score = sum of points of its alerts × chain multiplier, capped at 100.
- Chain multiplier by number of distinct kill-chain stages: 1 stage ×1.0, 2 stages ×1.3, 3+ stages ×1.6.
- Levels: 0–29 low, 30–59 medium, 60–79 high, 80–100 critical. Incident score uses the same formula over all its alerts.

Correlation algorithm

- Nodes = alerts. Link two alerts if they share an IP, a user or (for R3) a /24 subnet **and** are within 2 hours of each other.

- **IP→user pivot:** if IP X has an alert and there is a successful login of user U from X, alerts on U after that login are linked to X's alerts.
- Union-find over the links; each connected component is one incident. Only alerts are nodes, so normal events never merge incidents.

Attack story generation

- Sort incident alerts by time. Stage order: Reconnaissance → Credential Access → Initial Access → Privilege Escalation → Persistence → Exfiltration.
- Each rule has a sentence template filled with real values (IP, user, count, time span). Consecutive alerts of the same rule are merged into one sentence.
- Title and assessment are chosen from the stages present (e.g. Credential Access + Initial Access + Persistence → "SSH compromise with persistence"). Each rule contributes a recommended action.

Simulator scenarios (Bhanu writes attacks, Mahadev writes baseline)

ID	Scenario	Injected behaviour	Expected stages
S1	SSH breach	Brute force from one IP → success as deploy → sudo → useradd sysupdate	Cred. Access, Initial Access, Priv. Esc., Persistence
S2	Password spray	One IP tries 12 users × 2 attempts; one succeeds	Cred. Access, Initial Access
S3	Web recon to exfil	gobuster 404 flood → SQLi attempts on /login → 200 on /admin → 80 MB download	Recon, Initial Access, Exfiltration (possible)
S4	Low and slow	3 IPs in one /24, ~8 attempts each over 6 h (24 total, ~1 every 15 min). No single IP reaches 15, so only /24 aggregation catches it	Cred. Access
S5	Insider off-hours	Valid user logs in at 03:10 from new IP → sudo → large download	Initial Access, Priv. Esc., Exfiltration (possible)

Baseline traffic (Mahadev): 25 users logging in 08:00–20:00 from 1–2 fixed IPs each, occasional single typo failures, ~200 web clients browsing normal pages with a few 404s, over 3 days, fixed random seed. Output in the exact real log formats below. Attacks are appended by Bhanu's script, which also writes **labels.json** (malicious IPs, users, scenario stages).

auth.log sample:

```
Oct  4 02:03:11 web01 sshd[2211]: Failed password for invalid user admin
from 185.220.101.7 port 51122 ssh2
Oct  4 02:41:09 web01 sshd[2290]: Accepted password for deploy
from 185.220.101.7 port 51877 ssh2
Oct  4 02:43:30 web01 sudo: deploy : TTY=pts/0 ; USER=root ; COMMAND=/bin/bash
Oct  4 02:45:02 web01 useradd[2331]: new user: name=sysupdate, UID=1005
```

access.log sample (combined format):

```
45.33.10.9 - - [04/Oct/2026:14:02:10 +0530] "GET /login.php?id=1' OR '1'='1 HTTP/1.1"
200 512 "-" "sqlmap/1.8"
```

(Real logs have one event per line; wrapped here only for print.)

07 Work Division — Bhanu (complex) vs Mahadev

Rule: Bhanu owns everything where a bug breaks detection correctness. Mahadev owns everything the judges see and read. They meet only at the frozen data contract, so neither person waits for the other.

#	Bhanu Prasad — complex work	Done by
B1	Freeze schemas.py + API contract + mock JSON files (together with Mahadev)	H1
B2	Parsers for auth.log and access.log, format auto-detect, tolerant of bad lines, year/timezone handling	H3
B3	Attack injector for S1-S5 + labels.json	H4
B4	P0 rules: R1, R2, R4, R7, R8, R9, R10 + unit tests	H4-5
B5	Risk scoring + correlation engine (union-find, IP→user pivot)	H6
B6	Attack story generator, MITRE mapping, recommendations	H6:30
B7	PostgreSQL persistence (db.py, COPY bulk insert, read queries) + FastAPI endpoints wired to engine; serves frontend build	H7
B8	P1 rules: R3, R5, R6, R11 + baseline module	H8
B9	Evaluation module (precision/recall), edge-case tests, threshold tuning, performance test	H10
B10	Deployment, integration lead, demo rehearsal, backup video	H11-12

#	Mahadev H — frontend, data and docs	Done by
M1	Agree contract (H1); scaffold React + Vite + Tailwind + Router; api.js that can switch between mocks and real API with one flag	H1-2
M2	Baseline traffic generator (normal auth + web logs, fixed seed, exact formats) — hand to Bhanu by H2:30	H2:30
M3	Upload page (drag-drop, multiple files) + Simulation panel (scenario checkboxes, Run button)	H3
M4	Overview dashboard: stat cards, events-over-time chart, top risky entities	H4
M5	Incident list (cards with level badge, entities, stage chips)	H5
M6	Incident detail (highest UI priority): story panel, kill-chain stage bar, vertical timeline, evidence drawer with raw lines in monospace + line numbers	H6:30
M7	Entities table (search, sort, filter by level) + Evaluation page	H8
M8	Loading, error and empty states; API-down banner; manual UI test checklist	H10
M9	README, architecture diagram, slides (problem, solution, demo, innovation, results, limitations)	H11
M10	Record demo screen capture; final fresh-clone test with Bhanu	H11-12

UI pages Mahadev builds

Incident Detail is the star of the app — it gets the most polish. Overview stays simple: a few cards and one chart.

Page	Contents
Home / Upload	Drag-drop log files or “Run Attack Simulation” with S1-S5 checkboxes; shows parse stats after upload; “Recent analyses” list from GET /api/analyses (P1)

Page	Contents
Overview	Kept simple: cards for lines parsed / skipped, alerts, incidents by level; one events-over-time chart; top 5 risky entities
Incidents	Cards sorted by risk: title, level badge, score, IPs/users, stage chips, time span
Incident detail	Centerpiece. Header with level + risk score; attack story; kill-chain stage bar; timeline of steps; click a step → evidence drawer with raw log lines and rule reason; recommendation box
Entities	Table of IPs and users with risk score, level, alert count; search box and level filter
Evaluation	Headline: scenarios detected 5/5 with per-scenario ✓/✗; then precision, recall, critical false positives (simulated runs only)

08 12-Hour Build Plan (matches the official mission timeline)

Hour	Phase	Bhanu	Mahadev
1	Understand + Plan	Freeze schemas + API contract; write mock JSON; create repo; create local + managed Postgres DB and run schema.sql	Scaffold React app; set up api.js with mock switch; read the contract
2-4	Core Development	Parsers + auto-detect; attack injector; P0 rules R1, R2, R4, R7, R8 with tests	Baseline generator (handoff H2:30); Upload + Simulation page; Overview on mocks
	Checkpoint C1 (end H4)	CLI run on generated logs prints correct alerts for S1-S3	All pages so far render from mock JSON
5-8	Build + Integrate	R9, R10; scoring; correlation; story; FastAPI endpoints (H6:30); then P1 rules	Incident list + detail + evidence drawer; Entities; Evaluation shell
	H7: INTEGRATION	Both switch from mocks to real API together; fix contract mismatches immediately	
	Checkpoint C2 (end H8)	Full end-to-end works for S1-S5 via the UI. If not: cut all P2 and unfinished P1	
9-10	Testing + UI	Evaluation module; edge-case tests; tune thresholds to zero FPs on baseline; 100k-line perf test; first deploy (H9)	UI polish; loading/error/empty states; manual UI checklist; test on deployed URL
	End H10: FEATURE FREEZE	Only bug fixes from here	
11	Docs + Presentation	Known limitations section; demo script; rehearse; record backup video	README; architecture diagram; slides
	H11:30: CODE FREEZE		
12	Final Testing + Submission	Fresh clone → run tests → verify deployed demo → submit by T-00:20	Same checklist together; final slide check

Priority and cut list

Priority	Features	Rule
P0 — must ship	auth + access parsers; R1, R2, R4, R7, R8, R9, R10; scoring; correlation; story; upload; simulate; incidents list + detail + evidence	Demo is impossible without these. Done by H8.
P1 — should ship	R3, R5, R6, R11; baseline; entities table; overview chart; evaluation page	Build after C2 passes.
P2 — only if time	Isolation Forest anomaly score; generic JSON-lines format; export incident report; impossible travel	Never start after H9.

Emergency MVP (executable fallback)

Early warning — H6:30: if the engine CLI cannot print a correct S1 attack story, Bhanu tells Mahadev immediately.

Trigger — H8 (C2 fails): stop all new work and switch to the Emergency MVP. Do not try to finish everything.

Emergency MVP scope:

- Rules: R1, R2, R4, R7, R8 (plus R9, R10 only if already working)
- Scenarios: S1 and S2 fully; S3 as recon + initial access only (its exfil stage needs R11, which is P1)
- Pages: Upload/Simulate → Incident list → Incident detail with story, timeline and evidence
- Dropped: Entities page, Evaluation page (put pytest results and metrics on a slide instead), all P1/P2

Emergency demo: one complete, flawless S1 attack story — brute force → login → sudo → backdoor — with click-to-evidence.

09 Integration and Testing Plan

What we build must integrate and be tested. The strategy is **contract-first, mock-first**:

- Hour 1: schemas.py and mock JSON files (one analysis with 3 incidents) are committed. These are the single source of truth.
- Mahadev builds every page against the mocks; Bhanu builds the engine so its real output matches the mocks exactly.
- api.js has one flag USE MOCKS. At H7 it is flipped to false — integration is a switch, not a rewrite.
- Small commits to main every hour; each person works only in their own folders, so merge conflicts are near zero.

Definition of Done (applies to every feature)

- Merged to main, unit test passes (engine) or checklist item ticked (UI).
- Works through the UI against the real API, with no console errors.
- Handles empty and malformed input without crashing.

Test cases

ID	Test	Expected result	Owner
T1	Baseline only (no attacks)	0 critical incidents, at most 1 high — proves low false positives	Bhanu
T2	Each scenario S1-S5 injected separately	Exactly one incident with the expected entities and stages	Bhanu
T3	All 5 scenarios together	5 separate incidents — not merged into one	Bhanu
T4	Empty file	HTTP 400: “no parsable lines”; UI shows message	Both
T5	Binary / image file	Rejected with clear error, no crash	Both

ID	Test	Expected result	Owner
T6	20% malformed lines mixed in	Rest parsed; skipped count shown in UI	Bhanu
T7	Unknown log format	Error lists supported formats	Bhanu
T8	auth.log crossing 31 Dec → 1 Jan (no year in syslog)	Timeline order still correct	Bhanu
T9	IPv6 addresses	Parsed and detected like IPv4	Bhanu
T10	100,000 lines	Analysis completes in under 10 seconds	Bhanu
T11	Same file uploaded twice	Identical results (deterministic)	Bhanu
T12	Real-world Loghub OpenSSH sample	≥95% of lines parsed	Bhanu
T13	Evidence integrity	Every alert's line numbers point to the correct raw lines in the file	Bhanu
T14	Backend down	UI shows error banner, not a blank screen	Mahadev
T15	Full demo flow on deployed URL and on localhost	Upload → overview → incident → evidence → evaluation works on both	Both
T16	URL-encoded SQLi (%27%200R%20%271%27%3D%271)	R10 fires because requests are decoded before matching	Bhanu
T17	Same logs shuffled or split across two files	Identical incidents to the sorted single-file run	Bhanu
T18	Restart the backend, then open an old analysis	Incidents, story and evidence load from PostgreSQL unchanged	Bhanu
T19	Database unreachable during upload	Clear HTTP 503 “storage unavailable”; no partial rows (single transaction); UI shows banner	Both

Success targets (shown in this order on the Evaluation page and slides):

Scenarios detected 5/5
 Precision ≥ 90% (entity level)
 Recall ≥ 90% (entity level)
 Critical false pos. 0 (baseline-only data)
 100k lines < 10 s

10 How to Do It — What To Do and What Not To Do

DO	DON'T
✓ Freeze the data contract in Hour 1 and build against mocks.	✗ No LLM in the detection path — it is slow, non-deterministic and can hallucinate.
✓ Get P0 working end-to-end by Hour 8 before adding anything else.	✗ No Kafka, Elasticsearch, Redis, Alembic migrations or microservices — one Postgres, one schema.sql.
✓ Make every alert carry its evidence line numbers from day one.	✗ No login system for the app itself.
✓ Run detection in memory; write to PostgreSQL once, in one transaction, after analysis.	✗ No deep learning or model training marathons.
✓ Use a fixed random seed so demos are repeatable.	✗ No real-time streaming.
✓ Run the baseline-only test often to kill false positives.	✗ No third log format until P0 and P1 are done.

✓ Deploy early (Hour 9), not at Hour 12.	✗ Never change the API contract without telling the other person.
✓ Commit small and often; work only in your own folders.	✗ No refactoring after the Hour 10 feature freeze.
✓ Rehearse the demo twice and keep a backup video.	✗ Don't rely only on venue Wi-Fi — keep localhost ready.
✓ Disclose datasets and AI tools used in the README.	✗ Never scan or attack a real system; all data is synthetic or public.

11 Pros and Cons

Pros	Cons (and how we handle them)
Explainable and deterministic — every decision traces to a named rule and raw lines; judges can verify it.	Rules can miss novel attacks — mitigated by the per-user baseline and optional Isolation Forest (P2); stated in limitations.
Fits 12 hours — small, focused scope with a clear cut list.	Only two log formats — chosen because they are the most common; parsers are modular so new formats are easy later.
Measurable — precision/recall on labelled data, which most teams won't have.	Synthetic data can look “rigged” — we also test on the real Loghub OpenSSH sample and let judges pick scenarios live.
Strong demo — a live attack is injected and turned into a story in seconds.	Thresholds need tuning — all in one config file; tuned against baseline in Hour 9.
Fast and offline — no external APIs, works without internet.	auth.log has no year or timezone — handled with configurable defaults and a rollover test (T8).
Portfolio value — a mini SIEM with detection engineering and MITRE mapping is directly relevant to SOC and security roles.	Postgres adds setup and a failure point — engine is DB-independent; managed DB created in H1; local fallback; tests T18–T19. No auth or real-time (out of scope).
Clean split of work — Bhanu and Mahadev never block each other. Persistent history — analyses survive restarts and can be reopened.	MITRE mapping is approximate — we map only where we can justify it; exfiltration is mapped at tactic level.

12 Risks and Mitigations

Risk	Impact	Mitigation
Integration breaks at H7	High	Contract-first mocks; C1 check at H4; both present at the switch
Parser fails on judges' logs	High	Tolerant parsing, skip-and-count, auto-detect, Loghub test (T12)
False positives during live demo	High	Baseline test T1; config thresholds; tuning in H9; fixed seed
Running out of time	High	P0/P1/P2 cut list; hard cut decision at C2 (H8)
Deployment fails	Medium	Deploy at H9; localhost fallback; backup video
Mahadev blocked waiting on backend	Medium	Mocks + independent generator task
PostgreSQL unavailable (venue network / managed DB down)	Medium	Local Postgres installed and tested in H1; DATABASE_URL switch; engine works without DB for CLI demo
Separate attacks merged into one incident	Medium	2-hour window, alerts-only graph, test T3

13 3-Minute Demo Script

Time	What we show	What we say
0:00	Title slide	"One attacker is hiding in 50,000 log lines. Traditional alerting can leave analysts with hundreds of individual events. ChainTrace turns them into one attack story."
0:20	Simulation panel — judge picks scenarios	"Pick any attacks — we'll hide them in three days of normal traffic."
0:40	Overview	"50,000 lines → 5 incidents. Bad lines skipped and counted, not crashed on."
1:00	Critical incident detail	Read the attack story: brute force → login → sudo → backdoor, with MITRE stages.
1:40	Click a step → evidence drawer	"Every sentence is backed by the exact raw log lines."
2:00	Password-spray / low-and-slow incident	"A fixed per-IP threshold misses these. Subnet and multi-user aggregation catches them."
2:20	Evaluation page	"5 of 5 injected attacks detected, with precision, recall and zero critical false positives on clean traffic."
2:40	Architecture slide	"Simple, explainable, measurable — built to work, not just to look good."

Judge Q&A prep

Likely question	Our answer
"Where is the AI?"	We deliberately kept detection deterministic because security decisions must be explainable. Every detection traces to a specific rule, threshold and raw log line.
"How do you know this is exfiltration?"	We don't claim it is. We flag a possible large data transfer based on HTTP response volume in the access log, and say so in the story.
"Isn't your data rigged?"	You pick the attacks live, the baseline uses a fixed seed, and we also parse a real-world Loghub OpenSSH sample.
"Is that SQL injection confirmed?"	No — it is a matched signature on the decoded request. Confirmation would need application-side evidence.
"What happens with messy logs?"	Bad lines are skipped and counted, never crash the run; events are time-sorted, so unordered or split files give the same result.

14 Submission Checklist (official expectations)

Requirement	Our deliverable	Owner
Working project + deployed demo	Render/Railway URL + localhost instructions	Bhanu
Repository with clear README	Setup, run, test commands, screenshots	Mahadev
Architecture diagram + technical decisions	docs/architecture.png + "Why" section in README	Mahadev
Core workflow demonstration	3-minute live demo + backup video	Both
Testing evidence + edge cases	pytest output, T1-T19 results, evaluation metrics screenshot	Bhanu
Known limitations + future improvements	README section (see below)	Bhanu
Disclosure of APIs, datasets, AI assistance	README: Loghub sample, no external APIs, AI tools used during development	Mahadev

Known limitations and future improvements

- Supports two log formats; future: Windows Event Logs, firewall logs, JSON lines.
- Batch analysis only; future: streaming ingestion and live alerts.
- Rule-based core; future: ML anomaly scoring and learned thresholds.
- No user accounts; future: multi-user cases, analyst notes, retention policy for old analyses in PostgreSQL.
- No threat-intel enrichment; future: IP reputation and GeoIP for impossible-travel detection.

Remember: AI may help us build it, but we still have to make it work. A finished, tested, explainable ChainTrace beats an unfinished “AI-powered SIEM” every time.